TECH CHALLENGE — FASE 1

Sistema inteligente de suporte ao diagnóstico
e detecção de riscos em saúde da mulher

Detecção precoce de câncer de mama

Machine Learning sobre dados de exame (tabular)
+ Visão Computacional sobre imagens de ultrassom (CNN)

Item	Conteúdo
Autor(es)	Enio Rodrigues
Turma / Fase	IADT — Fase 1
Repositório Git	https://github.com/eniorodrigues/Tech_Challenge
Vídeo (YouTube)	https://youtu.be/XWGwS7_uU2o
Dataset principal	Breast Cancer Wisconsin (Diagnostic) — UCI / scikit-learn
Dataset EXTRA	BreastMNIST (ultrassom de mama, derivado do BUSI) — medmnist / Zenodo
Stack	Python 3.11 · scikit-learn · pandas · SHAP · PyTorch · Docker

Sumário

Seção	Conteúdo
1. Visão geral	O problema, a solução entregue e a premissa clínica
2. Dataset	Bases escolhidas, origem, licença e discussão do problema
3. Código-fonte	Estrutura do projeto e responsabilidade de cada módulo
4. Execução	Instruções locais e via Docker (Dockerfile incluído)
5. Relatório técnico	EDA · pré-processamento · modelos · resultados · explicabilidade · crítica
6. EXTRA — Visão Computacional	CNN em ultrassom, ponto de operação clínico e Grad-CAM
7. Provas de correção	Suíte de testes, com destaque para o teste anti-vazamento
8. Galeria de resultados	Todas as figuras geradas pelo pipeline
9. Conformidade	Mapa item-a-item do enunciado para a entrega
10. Vídeo	Roteiro e o que é demonstrado em cada bloco
Anexos	Dataset, reprodutibilidade e referências

1. Visão geral da solução

Uma rede de hospitais especializada no atendimento à mulher precisa **acelerar a triagem** e **apoiar a decisão médica** diante de um volume crescente de pacientes. Nesta Fase 1 o escopo é construir a **base de IA** do sistema: um modelo de Machine Learning capaz de analisar dados médicos automaticamente e sinalizar padrões de risco.

A solução entregue ataca a **detecção precoce de câncer de mama** — a tarefa principal sugerida no enunciado — por **duas vias complementares**:

- **Via tabular (obrigatória).** Classificação binária de tumores em malignos vs. benignos a partir de 30 medidas morfométricas de núcleos celulares, com quatro algoritmos comparados, validação cruzada, avaliação em holdout e explicabilidade por feature importance e SHAP.
- **Via de imagem (EXTRA).** Uma Rede Neural Convolucional que diagnostica diretamente de **imagens de ultrassom de mama**, com escolha explícita do ponto de operação clínico e explicabilidade visual por Grad-CAM.

> **Premissa central do projeto:** o modelo é um **copiloto**, nunca um substituto. Ele entrega uma probabilidade acompanhada de uma explicação; a conduta é sempre do profissional de saúde. **O médico tem a palavra final no diagnóstico.**

1.1 O que foi entregue

Entregável	Onde está	Status
Código-fonte completo	`src/`, `main.py`, `main_vision.py`, `tests/`	Completo
README com instruções	`README.md`	Completo
Dockerfile	`Dockerfile` + `.dockerignore`	Completo
Dataset	`data/breast_cancer_wisconsin.csv` + download automático	Completo
Resultados (gráficos e análises)	`reports/figures/` (11 figuras) + `reports/*.csv`	Completo

Entregável	Onde está	Status
Relatório técnico	Seção 5 deste PDF + notebooks narrados	Completo
Notebooks de demonstração	`notebooks/` (2 notebooks, com saídas salvas)	Completo
Roteiro do vídeo	Seção 10 deste PDF + `docs/ROTEIRO_VIDEO.md`	Completo

2. Dataset e discussão do problema

2.1 Base principal — Breast Cancer Wisconsin (Diagnostic)

São **569 pacientes** e **30 features numéricas** extraídas por processamento de imagem de aspirados por agulha fina (PAAF) de massas mamárias. Cada feature descreve uma propriedade geométrica ou de textura do núcleo celular — raio, textura, perímetro, área, suavidade, compacidade, concavidade, pontos côncavos, simetria e dimensão fractal — em três resumos por paciente: **média**, **erro-padrão** e o **pior** (maior) valor observado. Daí 10 × 3 = 30 colunas.

Atributo	Valor
Fonte original	UCI Machine Learning Repository (Wolberg, Street & Mangasarian, 1995)
Acesso no projeto	`sklearn.datasets.load_breast_cancer()` — embutida na biblioteca, sem download externo
Link Kaggle (mesma base)	kaggle.com/datasets/uciml/breast-cancer-wisconsin-data
Cópia versionada	`data/breast_cancer_wisconsin.csv` (569 linhas × 31 colunas)
Amostras	569 pacientes
Features	30 numéricas contínuas (nenhuma categórica, nenhum identificador)
Alvo	`diagnosis` — binário
Distribuição	357 benignos (62,7%) · 212 malignos (37,3%)
Valores ausentes	Nenhum

2.2 Por que esta base

- É a **tarefa principal explicitamente sugerida** no enunciado do desafio.
- É **pública, consolidada e livre de dados identificáveis** — não há questão ética de reidentificação de pacientes.
- Está **embutida no scikit-learn**, o que torna o projeto **100% reprodutível** em qualquer máquina, sem conta no Kaggle e sem link que possa cair.
- O problema é **clinicamente real e de alto impacto**: o câncer de mama é o câncer mais incidente em mulheres no Brasil, e a detecção precoce é o principal determinante de sobrevida.

2.3 Decisão de projeto — a convenção do rótulo

O scikit-learn carrega esta base com `target 0 = malignant` e `1 = benign`. Como as métricas binárias do scikit-learn (precision, recall, F1) são calculadas por padrão sobre a **classe 1**, manter essa codificação faria o *recall* reportado significar "proporção de **benignos** detectados" — exatamente o oposto do que interessa clinicamente.

Por isso o projeto **remapeia o alvo para 1 = maligno e 0 = benigno** em `src/data_loader.py`. Com isso, **recall passa a significar literalmente "proporção de tumores malignos que o sistema detectou"** — e todas as discussões de métrica deste relatório ficam diretamente interpretáveis. Existe um teste automatizado dedicado a provar que esse remapeamento está correto (`test_remapeamento_do_alvo`), porque um erro silencioso aqui inverteria todas as conclusões.

2.4 Base do item EXTRA — BreastMNIST (ultrassom)

Atributo	Valor
Base	BreastMNIST, do benchmark MedMNIST v2
Origem dos dados	BUSI — Breast Ultrasound Images Dataset (Al-Dhabyani et al., 2020)
Acesso no projeto	pacote `medmnist`, download automático do Zenodo (sem Kaggle)
Amostras	780 imagens de ultrassom em tons de cinza (546 treino / 78 validação / 156 teste)
Resolução usada	64 × 64 px, 1 canal
Alvo	binário — remapeado para **1 = maligno**, igual ao pipeline tabular

A escolha do BreastMNIST no lugar do CBIS-DDSM (sugerido no enunciado) foi deliberada: o CBIS-DDSM tem dezenas de gigabytes e exige credencial do Kaggle, o que quebraria a reprodutibilidade de um clone limpo do repositório. O BreastMNIST resolve o **mesmo problema clínico** — diagnóstico de câncer de mama a partir de imagem médica — com download automático e treino em CPU em poucos minutos.

3. Código-fonte — estrutura do projeto

O projeto é um **pacote Python estruturado**, não um notebook solto: a lógica vive em módulos testáveis em `src/`, os scripts de orquestração apenas os encadeiam, e os notebooks consomem os mesmos módulos. Isso garante que o que é demonstrado no notebook é exatamente o que roda em produção.

```
Tech_Challenge/
|-- main.py                  Pipeline tabular completo (CLI, 7 etapas)
|-- main_vision.py           EXTRA: pipeline da CNN de imagem (CLI)
|-- Dockerfile               Imagem reprodutivel (CPU)
|-- requirements.txt         Dependencias com versoes fixas
|-- README.md                Instrucoes de execucao e metodologia
|-- src/
|   |-- config.py            Semente, caminhos, convencao de rotulo
|   |-- data_loader.py       Carga da base + remapeamento do alvo
|   |-- preprocessing.py     Limpeza, split estratificado, scaler (sem leakage)
|   |-- eda.py               Analise exploratoria e graficos
|   |-- modeling.py          4 modelos + validacao cruzada
|   |-- evaluation.py        Metricas, matriz de confusao, curvas ROC
|   |-- explainability.py    Feature importance + SHAP
|   `-- vision.py            EXTRA: CNN, treino, avaliacao e Grad-CAM
|-- notebooks/
|   |-- tech_challenge_breast_cancer.ipynb   Notebook tabular (com saidas)
|   `-- tech_challenge_cnn_imagem.ipynb      EXTRA: notebook da CNN
|-- tests/
|   |-- test_pipeline.py     Provas de correcao tabular (inclui anti-leakage)
|   `-- test_vision.py       Provas de correcao da CNN (inclui remap do rotulo)
|-- docs/ROTEIRO_VIDEO.md    Roteiro do video de demonstracao
|-- data/                    CSV da base + BreastMNIST em cache
|-- models/                  melhor_modelo.joblib + cnn_breastmnist.pt (gerados)
`-- reports/
    |-- metricas_teste.csv
    |-- metricas_validacao_cruzada.csv
    `-- figures/             Todos os graficos em PNG (01 a 11)
```

3.1 Responsabilidade de cada módulo

Módulo	Responsabilidade	Ponto-chave
`config.py`	Semente única, caminhos e convenção de rótulo	`RANDOM_STATE = 42` em um só lugar garante reprodutibilidade
`data_loader.py`	Carrega a base e remapeia o alvo	Inverte o rótulo para 1 = maligno
`preprocessing.py`	Limpeza, imputação, padronização e split	O scaler vive dentro do `Pipeline` — sem vazamento
`eda.py`	Balanceamento, distribuições, correlação	Detecta pares com $\|r\| \geq 0{,}9$ (multicolinearidade)
`modeling.py`	Define os 4 modelos e a validação cruzada	`class_weight='balanced'` trata o desbalanceamento
`evaluation.py`	Métricas, matriz de confusão, curvas ROC	Expõe TN/FP/FN/TP para a discussão clínica
`explainability.py`	Feature importance e SHAP	Detecta automaticamente coeficiente vs. importância de impureza
`vision.py`	CNN, treino, limiar clínico e Grad-CAM	Calibra o limiar na validação, nunca no teste

4. Como executar

Requisito: **Python 3.11** (ou apenas Docker).

4.1 Execução local

```
# 1. Clonar e entrar no projeto
git clone https://github.com/eniorodrigues/Tech_Challenge.git
cd Tech_Challenge

# 2. Ambiente virtual + dependencias
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate        # Linux / macOS
pip install -r requirements.txt

# 3. Pipeline tabular -> figuras 01-07, metricas e o melhor modelo
python main.py

# 4. EXTRA: pipeline da CNN -> baixa o BreastMNIST, treina,
#    gera as figuras 08-11 e o modelo cnn_breastmnist.pt (CPU, ~2-3 min)
python main_vision.py

# 5. Provas de correcao
python -m pytest -v

# 6. Notebooks narrados
jupyter notebook notebooks/tech_challenge_breast_cancer.ipynb
jupyter notebook notebooks/tech_challenge_cnn_imagem.ipynb
```

As rodas CPU do PyTorch instalam mais rápido pelo índice dedicado: `pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu`.

4.2 Execução via Docker

O `Dockerfile` incluído no repositório monta a imagem com as rodas **CPU** do PyTorch (sem dependência de GPU/CUDA) e já deixa o pipeline pronto para rodar. Os volumes trazem as figuras e os modelos gerados de volta ao host.

```
# Build da imagem
docker build -t tech-challenge-fase1 .

# Pipeline tabular (comando padrao da imagem)
docker run --rm -v "$(pwd)/reports:/app/reports" \
                -v "$(pwd)/models:/app/models" tech-challenge-fase1

# Pipeline da CNN (EXTRA)
docker run --rm -v "$(pwd)/reports:/app/reports" \
                -v "$(pwd)/models:/app/models" \
                tech-challenge-fase1 python main_vision.py

# Provas de correcao
docker run --rm tech-challenge-fase1 python -m pytest -v

# Notebooks em http://localhost:8888
docker run --rm -p 8888:8888 tech-challenge-fase1 \
  jupyter notebook --ip=0.0.0.0 --port=8888 --no-browser --allow-root
```

4.3 Saídas geradas

Arquivo	Origem	Conteúdo
`reports/figures/01-07*.png`	main.py	EDA, matrizes de confusão, curvas ROC, importância de features e SHAP
`reports/figures/08-11*.png`	main_vision.py	Curvas de treino da CNN, matriz de confusão, amostras e Grad-CAM
`reports/metricas_teste.csv`	main.py	Métricas dos 4 modelos no conjunto de teste
`reports/metricas_validacao_cruzada.csv`	main.py	Média e desvio das métricas na validação cruzada de 5 folds
`models/melhor_modelo.joblib`	main.py	Pipeline treinado do melhor modelo (pré-processador + classificador)
`models/cnn_breastmnist.pt`	main_vision.py	Pesos da CNN de ultrassom

5. Relatório técnico

5.1 Discussões da análise exploratória

5.1.1 Balanceamento das classes

A base tem **357 casos benignos (62,7%) contra 212 malignos (37,3%)**. É um desbalanceamento **moderado** — não é o cenário extremo de fraude ou doença rara — mas já é suficiente para tornar a **accuracy uma métrica enganosa**: um classificador degenerado que respondesse "benigno" para toda paciente atingiria **62,7% de acurácia** sem detectar um único câncer. Essa constatação, feita logo na exploração, é o que motiva toda a escolha de métrica discutida na seção 5.4.

O desbalanceamento foi tratado em dois pontos: **estratificação** no split de treino/teste e na validação cruzada (para que a proporção de malignos seja idêntica em todos os conjuntos) e **ponderação de classe** (`class_weight='balanced'`) nos modelos que a suportam — o que penaliza mais o erro sobre a classe minoritária sem precisar reamostrar dados.

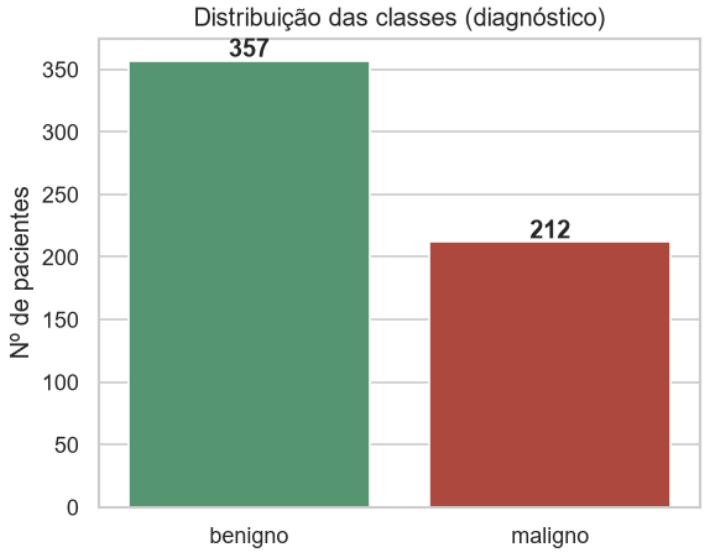


Figura 1 — Distribuição das classes. O desbalanceamento moderado (63/37) é o que invalida a accuracy como métrica isolada.

5.1.2 Escalas e estatísticas descritivas

As 30 features vivem em **ordens de grandeza radicalmente diferentes**: `mean area` varia na casa das centenas a milhares (≈ 143 a 2 501), enquanto `mean smoothness` se move entre 0,05 e 0,16. Sem padronização, qualquer algoritmo baseado em distância — o KNN é o caso óbvio — seria dominado quase inteiramente pelas colunas de área e perímetro, e a regressão logística teria coeficientes incomparáveis entre si. **Essa é a justificativa empírica direta para o StandardScaler** descrito na seção 5.2.

A varredura de valores ausentes retornou **zero** em todas as 31 colunas, e não há duplicatas exatas. A base é limpa — o que não elimina a necessidade do tratamento defensivo descrito adiante.

5.1.3 Padrão clínico encontrado — o achado central

Comparando as distribuições das seis features mais correlacionadas com o diagnóstico, emerge um padrão nítido e **clinicamente coerente**: as classes formam duas distribuições visivelmente deslocadas, com **zona de sobreposição estreita**. Em `worst concave points`, os casos benignos concentram-se abaixo de 0,10 e os malignos acima de 0,13; em `worst perimeter`, benignos ficam em torno de 85 e malignos se espalham de 110 a 250.

Leitura clínica: tumores malignos apresentam **núcleos celulares maiores** (raio, perímetro e área elevados) e **contornos mais irregulares** (concavidade e pontos côncavos elevados). É exatamente o critério morfológico que um patologista usa ao avaliar uma lâmina. O modelo, portanto, não vai precisar inventar um padrão exótico — ele vai aprender o critério que a medicina já reconhece, o que é a melhor notícia possível para a confiança clínica no sistema.

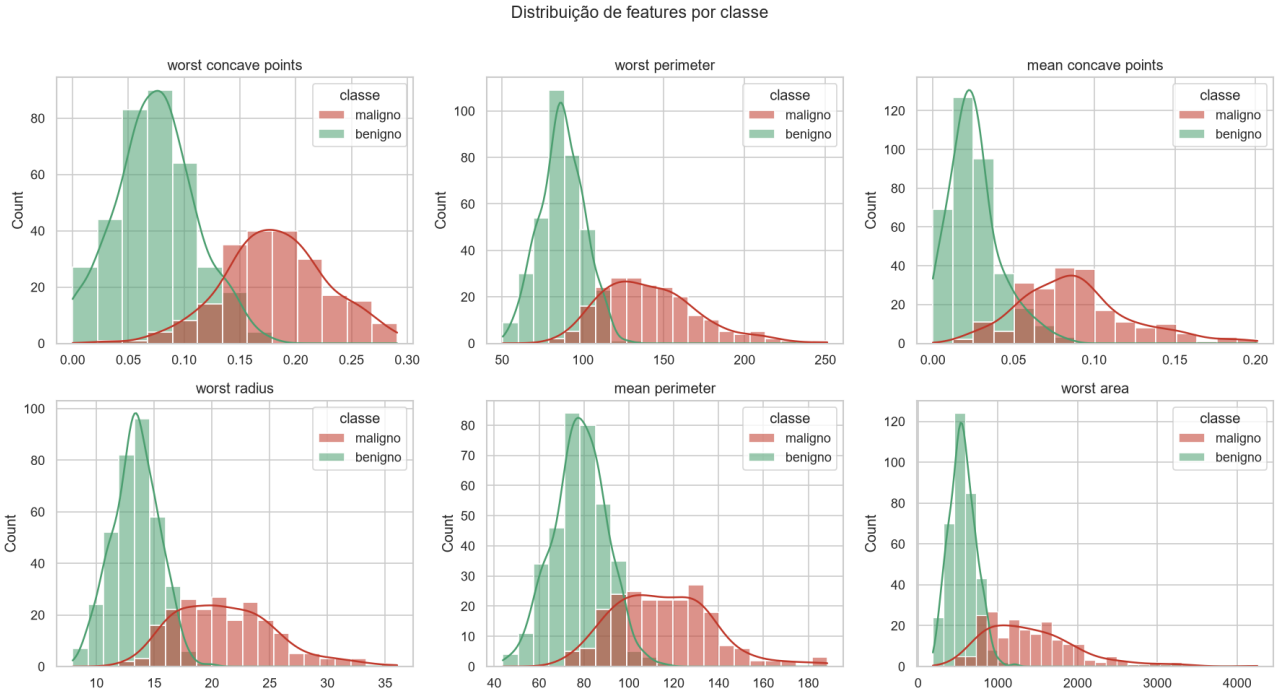


Figura 2 — Distribuição das seis features mais preditivas por classe (verde = benigno, vermelho = maligno). A separação clara com sobreposição estreita é o que explica o alto desempenho obtido.

5.1.4 Análise de correlação e multicolinearidade

A matriz de correlação de Pearson revela blocos densos de correlação altíssima. O pipeline identificou **21 pares de features com |r| ≥ 0,9**. Isso não é ruído nem erro de dados: é **redundância geométrica esperada** — raio, perímetro e área de um núcleo aproximadamente circular são transformações determinísticas umas das outras (perímetro ≈ $2\pi r$, área ≈ πr^2). O mesmo vale entre os blocos “mean” e “worst” da mesma medida.

Consequências práticas dessa multicolinearidade:

- **Não prejudica** o poder preditivo de árvores, Random Forest ou KNN.
- **Desestabiliza os coeficientes** da regressão logística: quando várias colunas carregam a mesma informação, o peso se divide arbitrariamente entre elas. Por isso **coeficiente linear não é medida confiável de importância** nesta base — e é exatamente por isso que o projeto usa SHAP como segunda lente (ver seção 5.5).
- **Justifica manter as 30 features** em vez de podar as redundantes: como o objetivo é desempenho preditivo e não inferência causal sobre coeficientes, remover colunas só traria risco de perder sinal marginal sem ganho prático.

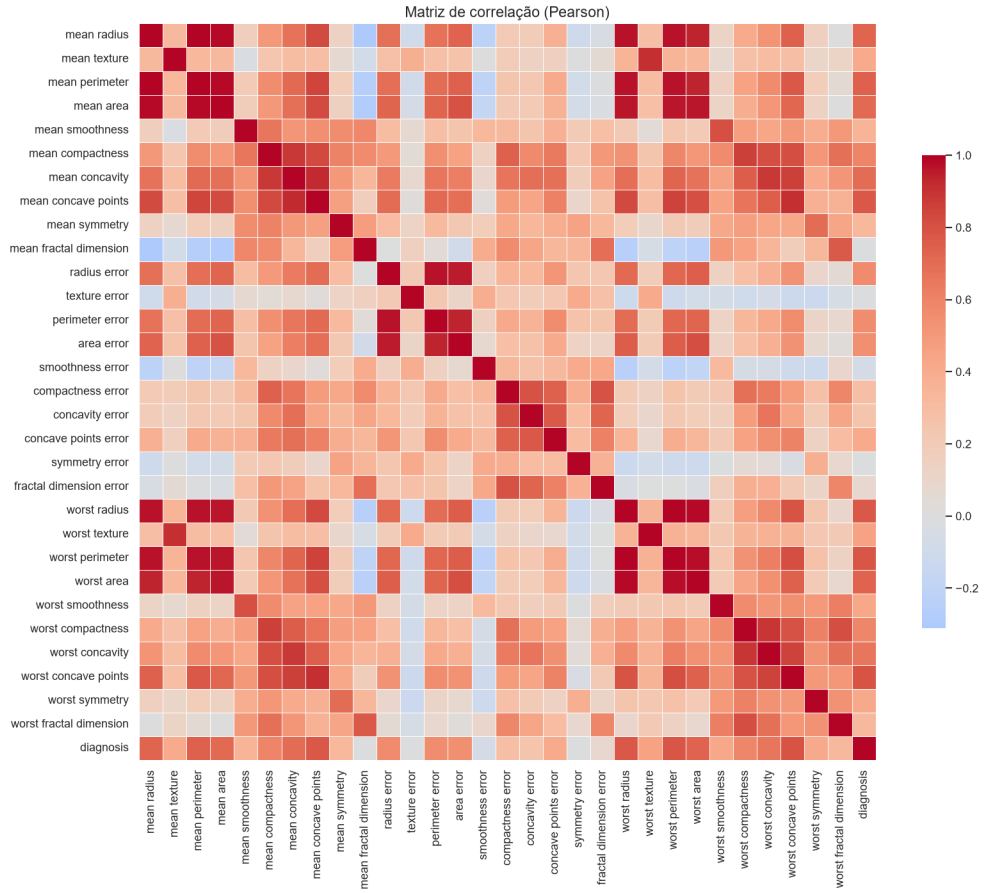


Figura 3 — Matriz de correlação de Pearson das 30 features e do alvo. Os blocos vermelhos escuros são a multicolinearidade geométrica descrita acima.

5.2 Estratégias de pré-processamento

Toda a estratégia foi desenhada em torno de um objetivo inegociável: **produzir uma estimativa honesta de desempenho**. Em saúde, um número inflado por vazamento de dados não é apenas um erro técnico — é uma promessa falsa sobre segurança de pacientes.

5.2.1 Limpeza defensiva

`clean_data()` remove duplicatas exatas e colunas integralmente vazias. Nesta base específica nenhuma das duas ocorre — a limpeza é **defensiva por design**, para que o mesmo pipeline possa ser reaproveitado em bases mais sujas (o dataset de SOP/PCOS sugerido no enunciado, por exemplo) sem reescrita.

5.2.2 Valores ausentes — imputação pela mediana, dentro do pipeline

A imputação usa `SimpleImputer(strategy='median')`. A mediana foi escolhida no lugar da média porque várias features são **assimétricas à direita com outliers pesados** (área e perímetro dos tumores grandes), e a média seria arrastada por eles. Criticamente, a imputação **não acontece na limpeza**: ela vive dentro do `Pipeline`, porque a mediana usada para preencher um valor faltante do teste precisa ser a mediana **do treino**.

5.2.3 Padronização e a prevenção de vazamento de dados

Esta é a decisão técnica mais importante do projeto. O `StandardScaler` (média 0, desvio 1) é obrigatório pelas razões de escala da seção 5.1.2 — mas **o modo como ele é aplicado** é o que separa um resultado válido de um resultado inflado.

O que o projeto NÃO faz: normalizar o DataFrame inteiro e só depois dividir em treino e teste. Isso é o erro mais comum em projetos de ML — e é vazamento: a média e o desvio calculados incluiriam as amostras de teste, de forma que o modelo teria acesso indireto a informação do conjunto que deveria ser inédito.

O que o projeto faz: encapsula imputador e scaler em um `ColumnTransformer` que, por sua vez, é a primeira etapa de um `Pipeline` do scikit-learn junto com o classificador. Assim, a cada `fit` — incluindo **cada um dos 5 folds da validação cruzada** — as estatísticas são reaprendidas **somente na partição de treino daquele fold** e apenas aplicadas à partição de validação.

```
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler",  StandardScaler()),
])
preprocessor = ColumnTransformer([("num", numeric_pipeline, numeric_features)])

# O classificador entra no MESMO pipeline -> o scaler nunca ve o teste.
model = Pipeline([("preprocessor", preprocessor), ("classifier", estimator)])
```

Essa invariante não fica no plano da intenção: existe um teste automatizado (`test_scaler_ajustado_somente_no_treino`) que **prova** que a média aprendida pelo scaler é idêntica à média do treino e mensuravelmente diferente da média do dataset completo. Ver seção 7.

5.2.4 Variáveis categóricas e numéricas

O enunciado pede a conversão de variáveis categóricas e numéricas para formatos adequados à modelagem. Nesta base, **as 30 features são todas numéricas contínuas** e o alvo é binário — não há nenhuma coluna categórica, textual ou de identificação a converter. O tratamento numérico exigido é justamente a imputação + padronização descrita acima.

Ainda assim, o pré-processador foi construído como `ColumnTransformer` — e não como um `Pipeline` simples — **precisamente para deixar o ramo categórico plugável**: acrescentar um `OneHotEncoder` para

colunas categóricas é uma linha, sem tocar em nenhum outro módulo. É a escolha arquitetural correta para um sistema que a rede hospitalar vai estender com prontuários reais (onde haverá sexo, comorbidade, histórico familiar).

5.2.5 Separação treino/teste

Parâmetro	Valor	Justificativa
Proporção	80% treino / 20% teste	455 amostras de treino e 114 de teste — teste grande o bastante para uma estimativa estável nesta base pequena
Estratificação	`stratify=y`	Mantém ≈ 37% de malignos nos dois conjuntos; sem isso, o teste poderia ficar com proporção distorcida
Semente	`random_state=42`	Reprodutibilidade exata; definida uma única vez em `config.py`
Validação cruzada	StratifiedKFold, 5 folds	Roda **apenas no treino**, para seleção de modelo; o teste permanece intocado até a avaliação final

5.3 Modelos usados e por quê

O enunciado exige duas ou mais técnicas. O projeto usa **quatro**, escolhidas deliberadamente para cobrir **famílias algorítmicas distintas** — linear, baseada em regras, baseada em distância e ensemble. Comparar famílias diferentes (em vez de quatro variações do mesmo algoritmo) é o que permite afirmar algo sobre a natureza do problema, e não apenas sobre um hiperparâmetro.

Modelo	Família	Por que foi incluído	Configuração
Regressão Logística	Linear	Baseline forte e **diretamente interpretável**: fornece probabilidade calibrada e coeficientes com sinal, o que facilita a conversa clínica. Adequada porque a EDA mostrou classes quase linearmente separáveis.	`max_iter=5000`, `class_weight='balanced'`
Árvore de Decisão	Baseada em regras	Máxima interpretabilidade — a decisão vira um fluxograma legível por um médico. Serve também como **piso de referência**: se um modelo complexo não superar uma árvore rasa, a complexidade não se justifica.	`max_depth=4` (limita overfitting), `class_weight='balanced'`
KNN	Baseada em distância	Não assume forma funcional alguma: classifica por semelhança com pacientes próximos — uma analogia natural ao raciocínio por casos parecidos. É também o teste empírico de que a padronização funciona.	`n_neighbors=7`
Random Forest	Ensemble de árvores	Costuma ser o mais forte em dados tabulares, captura interações não lineares, é robusto a outliers e fornece importância de features nativa — além de ser o modelo sobre o qual o SHAP é calculado de forma exata.	`n_estimators=300`, `class_weight='balanced'`

5.3.1 Protocolo de treino e seleção

- Cada modelo é um `Pipeline` completo (pré-processador + classificador) — nunca um estimador nu.
- **Seleção** por validação cruzada estratificada de 5 folds **no conjunto de treino**, com accuracy, precision, recall, F1 e ROC-AUC.
- **Avaliação final** uma única vez no holdout de teste, que não participou de nenhuma decisão anterior.
- **Critério de escolha do campeão:** maior **recall** da classe maligna — a métrica justificada na seção 5.4.

5.3.2 Desempenho na validação cruzada (conjunto de treino)

Modelo	Accuracy	Precision	Recall	F1	ROC-AUC
Regressão Logística	0,971 ± 0,018	0,965 ± 0,029	0,959 ± 0,030	0,962 ± 0,024	0,995 ± 0,005
Random Forest	0,963 ± 0,015	0,948 ± 0,021	0,953 ± 0,030	0,950 ± 0,020	0,990 ± 0,006
Árvore de Decisão	0,932 ± 0,019	0,894 ± 0,029	0,929 ± 0,044	0,910 ± 0,025	0,928 ± 0,030
KNN	0,969 ± 0,008	0,988 ± 0,015	0,929 ± 0,024	0,957 ± 0,012	0,986 ± 0,014

Os desvios-padrão baixos (≤ 0,03 na maioria) indicam que o desempenho é **estável entre partições** — não é um resultado de sorte de um split específico. Regressão Logística e Random Forest já se destacam aqui, antecipando o resultado do teste.

5.4 Resultados e interpretação dos dados

5.4.1 A escolha da métrica — o custo assimétrico do erro

Antes de qualquer número, é preciso definir **o que conta como sucesso**. Neste problema os dois tipos de erro têm consequências **profundamente assimétricas**:

Tipo de erro	O que significa clinicamente	Consequência	Custo
Falso negativo	Um tumor **maligno** é classificado como benigno	A paciente é liberada sem investigação. O câncer segue evoluindo e o tratamento começa em estágio mais avançado — ou não começa.	**Altíssimo — risco à vida**
Falso positivo	Um tumor **benigno** é classificado como maligno	A paciente passa por biópsia e exames adicionais, com ansiedade e custo ao sistema, até ser descartado.	Real, porém muito menor

Disso decorre a decisão metodológica do projeto: **a métrica prioritária é o recall da classe maligna** — a fração de cânceres que o sistema efetivamente detecta. F1 e precision entram como **contrapeso**, para impedir a degeneração oposta (um modelo que grita "maligno" para todo mundo teria recall perfeito e seria clinicamente inútil). A **accuracy é reportada mas não decide**, pelo motivo já demonstrado na EDA: 62,7% é o piso trivial nesta base. O **ROC-AUC** complementa por medir a qualidade do ranqueamento independentemente do limiar escolhido.

5.4.2 Desempenho no conjunto de teste (114 pacientes)

Modelo	Accuracy	Precision	Recall	F1	ROC-AUC
Regressão Logística	0,974	0,976	**0,952**	0,964	0,995
Random Forest	0,974	1,000	0,929	0,963	0,998
KNN	0,956	0,974	0,905	0,938	0,982
Árvore de Decisão	0,930	0,925	0,881	0,902	0,901

Linha destacada: modelo campeão pelo critério de recall.

5.4.3 Interpretação — o trade-off entre os dois melhores

O conjunto de teste contém **114 pacientes: 72 benignos e 42 malignos**. Traduzindo as métricas em contagens de pacientes reais, que é como a discussão clínica precisa ser feita:

	Regressão Logística	Random Forest
Malignos detectados (VP)	**40 de 42**	39 de 42
Falsos negativos (câncer não detectado)	**2**	3
Falsos positivos (biópsia desnecessária)	1	0
Accuracy	0,974	0,974
Recall (maligno)	**0,952**	0,929
Precision	0,976	**1,000**
ROC-AUC	0,995	**0,998**

Os dois modelos empatam em accuracy (0,974) — e é justamente aí que a accuracy mostra por que não serve como critério: **ela não distingue os dois erros**. Descendo ao nível de paciente, a diferença é nítida e tem uma direção clara:

- O **Random Forest** tem **precision perfeita**: nenhum alarme falso. Todo paciente que ele apontou como maligno era realmente maligno. Em contrapartida, **deixou passar 3 cânceres**.
- A **Regressão Logística** deixa passar **2 cânceres** e gera **1 alarme falso**.
- **O trade-off, em termos concretos:** trocar 1 falso negativo por 1 falso positivo significa trocar **um câncer não detectado** por **uma biópsia desnecessária**. Pelo critério de custo assimétrico estabelecido em 5.4.1, essa troca é claramente favorável.

Modelo escolhido: Regressão Logística. Além do recall superior, ela traz três vantagens práticas: fornece **probabilidades bem calibradas** (essencial para ajustar o limiar de triagem), é o modelo **mais simples e auditável** dos quatro (um requisito real em contexto regulatório de saúde) e tem **custo computacional desprezível** em inferência.

Vale registrar a ressalva estatística honesta: com 42 malignos no teste, a diferença de 1 falso negativo entre os dois modelos **não é estatisticamente significativa** — o intervalo de confiança dessas taxas é largo em uma amostra desse tamanho. A escolha é defensável e coerente com o critério declarado, mas a decisão final entre os dois exigiria validação em uma base maior.

5.4.4 Desempenho dos demais modelos

- **KNN (recall 0,905; AUC 0,982):** desempenho respeitável, confirmando que a padronização funcionou — sem ela, o KNN colapsaria. Fica atrás por não modelar explicitamente a fronteira de decisão.
- **Árvore de Decisão (recall 0,881; AUC 0,901):** claramente a mais fraca, especialmente em AUC. Uma única árvore com profundidade 4 produz uma fronteira em degraus grosseiros, incapaz de acompanhar a separação suave observada na EDA. É exatamente o contraste que justifica o ensemble: o Random Forest, feito de árvores, salta de 0,901 para 0,998 de AUC.

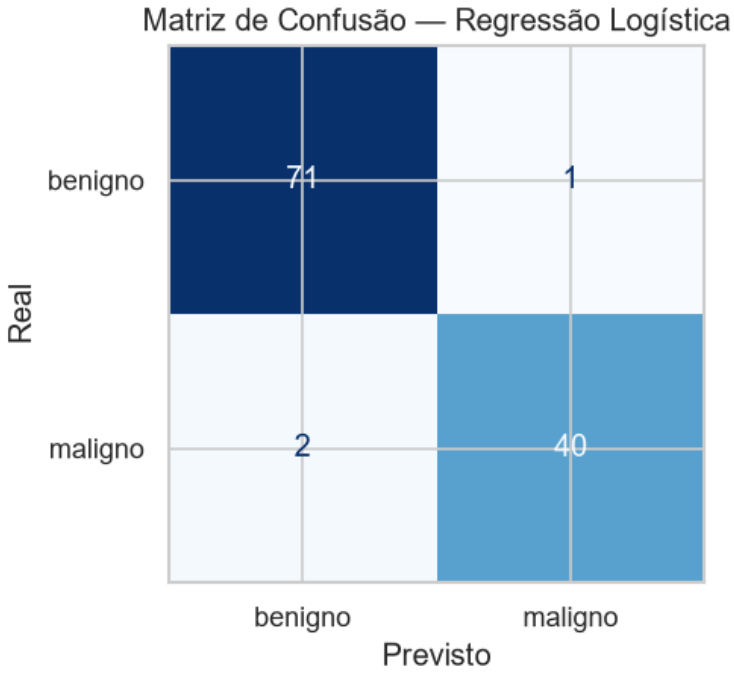


Figura 4 — Matriz de confusão do modelo campeão (Regressão Logística). O quadrante inferior esquerdo — 2 malignos classificados como benignos — é o erro que o projeto prioriza minimizar.

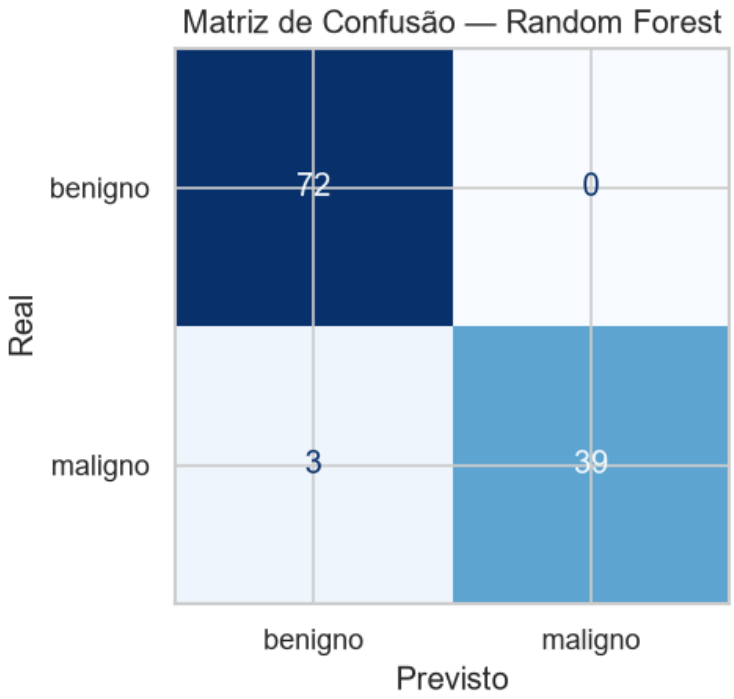


Figura 5 — Matriz de confusão do Random Forest: nenhum falso positivo, mas 3 falsos negativos.

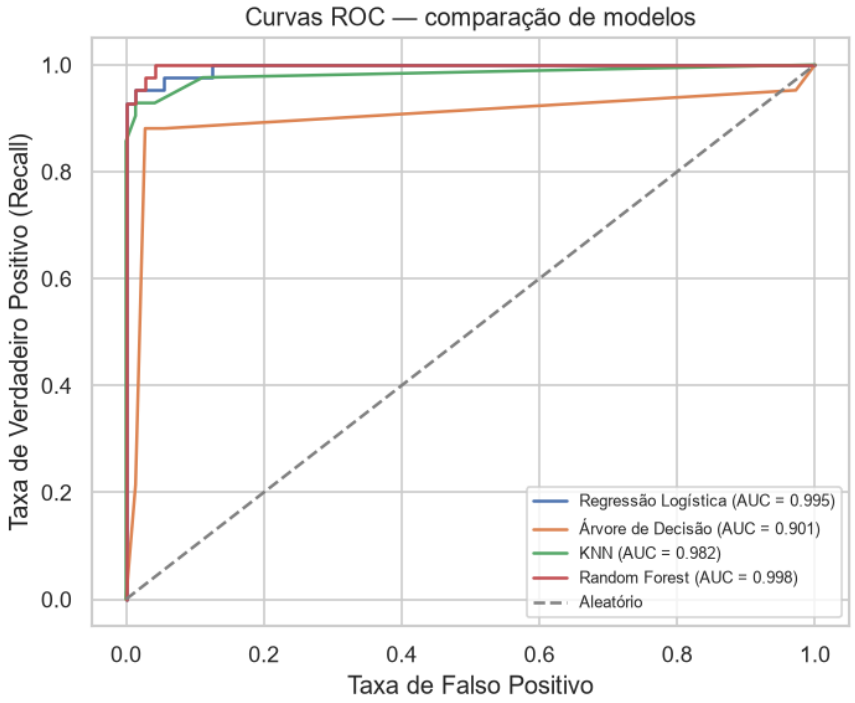


Figura 6 — Curvas ROC dos quatro modelos. Regressão Logística, Random Forest e KNN colam no canto superior esquerdo; a Árvore de Decisão é visivelmente inferior (AUC 0,901).

5.5 Explicabilidade — por que o modelo decidiu assim

Em saúde, um modelo que acerta mas não explica é inutilizável: nenhum profissional assume responsabilidade sobre uma recomendação opaca, e nenhum órgão regulador aprovaria. O projeto usa **duas técnicas complementares**, exatamente como pede o enunciado.

5.5.1 Feature importance — e uma armadilha honesta

Para a Regressão Logística, a importância global é o **valor absoluto dos coeficientes** — comparável entre features porque todas estão padronizadas. O topo traz `worst texture`, `radius error` e `worst symmetry`.

Esse resultado precisa ser lido com cuidado — e a leitura ingênua estaria errada. Ele **não** significa que raio, área e perímetro sejam irrelevantes (a EDA mostrou que são as features mais correlacionadas com o diagnóstico). É a **multicolinearidade** diagnosticada na seção 5.1.4 agindo: quando 21 pares de features carregam informação quase idêntica, o modelo linear **reparte o peso entre elas** e nenhuma sobressai isoladamente; sobrevivem no topo justamente as features **menos redundantes**, que contribuem com sinal próprio. **Coeficiente linear sob multicolinearidade não é uma medida confiável de importância** — e é precisamente por isso que o SHAP não é um extra decorativo neste projeto, mas a lente necessária.

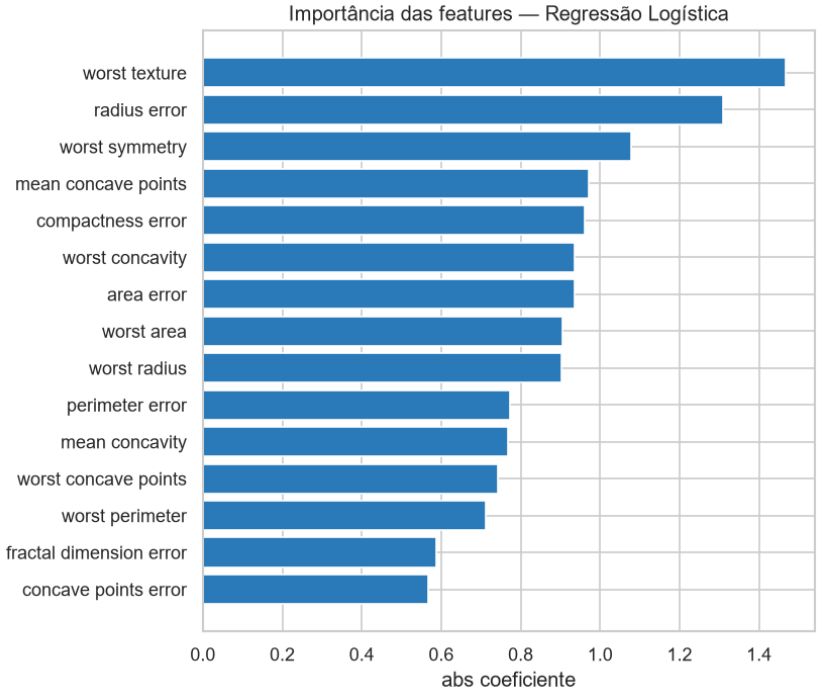


Figura 7 — Importância global por magnitude dos coeficientes (Regressão Logística). O ranking reflete a multicolinearidade da base, não a irrelevância das medidas de tamanho.

5.5.2 SHAP — a explicação que resiste à multicolinearidade

O SHAP (SHapley Additive exPlanations) tem base em teoria dos jogos: atribui a cada feature a **contribuição marginal média** para a previsão, considerando todas as ordens possíveis de inclusão. Isso o torna robusto à redundância e — mais importante — permite explicar **uma previsão individual**: "por que *esta* paciente foi classificada como maligno?". É a forma de explicação que efetivamente serve ao médico à beira do leito. Foi calculado com `TreeExplainer` sobre o Random Forest, onde a atribuição é exata.

O que o gráfico SHAP mostra:

- As features de maior impacto são `worst area`, `worst perimeter`, `worst concave points`, `mean concave points` e `worst radius` — **convergindo com a EDA** e corrigindo a leitura enviesada dos

coeficientes lineares.

- A **direção** é inequívoca: pontos vermelhos (valores altos dessas medidas) caem todos do lado positivo do eixo, **empurrando a previsão para "maligno"**; pontos azuis (valores baixos) empurram para "benigno".
- A separação é **limpa e monotônica**, sem a mistura desordenada que denunciaria o modelo se apoiando em artefato ou ruído.

Conclusão de explicabilidade: o modelo aprendeu que **núcleos celulares maiores e de contorno mais irregular indicam malignidade** — o mesmo critério morfológico da patologia clínica. Não é uma correlação espúria de dataset. Essa convergência entre o que o modelo usa e o que a medicina reconhece é o principal argumento a favor de confiar nele como ferramenta de apoio.

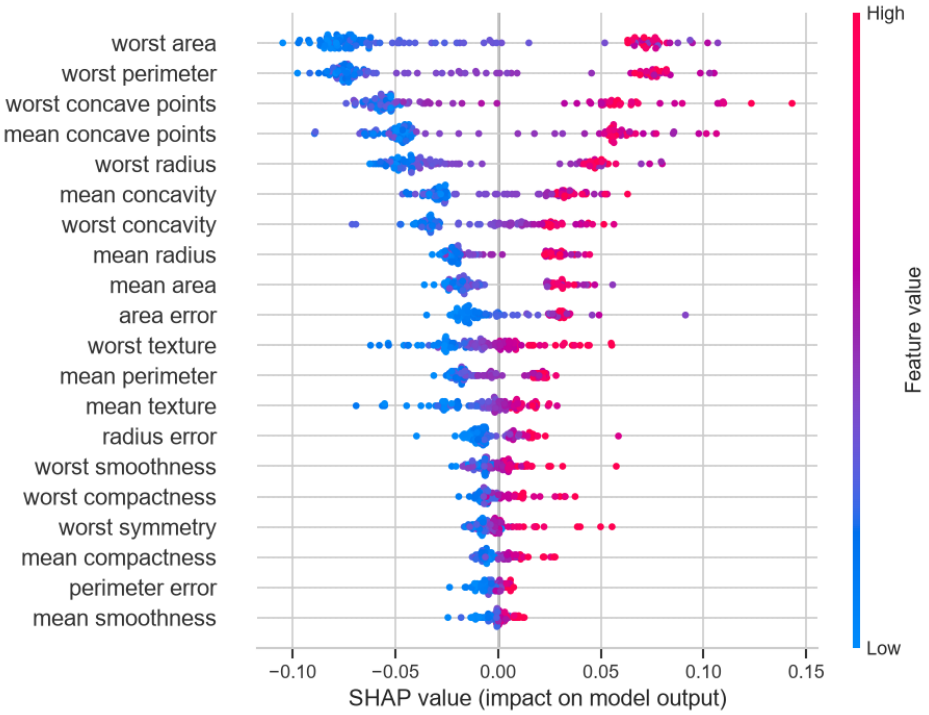


Figura 8 — SHAP summary plot (Random Forest). Cada ponto é uma paciente; a cor é o valor da feature e a posição horizontal é o empurrão na previsão. Valores altos (vermelho) de área, perímetro e pontos côncavos empurram consistentemente para "maligno".

5.6 Discussão crítica — o modelo pode ser usado na prática?

Resposta curta: sim, como ferramenta de apoio à decisão — e em nenhuma hipótese como decisor autônomo.

5.6.1 Como o sistema entraria no fluxo de atendimento

Uso	Como funciona	Ganho para a rede hospitalar
Priorização de fila	O sistema pontua os casos que chegam e sinaliza os de alto risco, que sobem na fila de leitura do especialista.	Reduz o tempo até o diagnóstico nos casos em que o tempo mais importa — o objetivo de triagem declarado no desafio.
Segunda opinião / rede de segurança	O modelo roda em paralelo ao laudo humano e emite um alerta apenas quando diverge, convidando a uma revisão.	Reduz a chance de um caso maligno passar por desatenção, sem adicionar trabalho aos casos concordantes.
Ponto de operação ajustável	Em triagem, opera-se com limiar mais baixo, favorecendo recall; em confirmação, com limiar mais alto.	A mesma ferramenta serve a duas etapas do fluxo com perfis de erro diferentes.

Em todos os cenários, a saída entregue é **probabilidade + explicação** (as features que mais pesaram naquele caso, via SHAP), nunca um veredito isolado. **A conduta é do profissional; o médico tem a palavra final.**

5.6.2 Limitações — leitura honesta

- **Base pequena e de fonte única.** 569 pacientes de uma única instituição, coletados nos anos 1990. Não há garantia de que o desempenho se mantenha em outra população, outro equipamento ou outro protocolo de coleta.
- **As features já foram extraídas por especialistas.** O modelo recebe medidas morfométricas prontas, calculadas a partir de lâminas de PAAF por um profissional. **Ele não substitui o exame** — ele opera *depois* dele. O item EXTRA de visão computacional é justamente a tentativa de encurtar essa dependência.
- **O teste tem 114 pacientes.** Diferenças de 1 a 2 erros entre modelos estão dentro da margem de incerteza amostral, como registrado em 5.4.3.
- **Não há informação demográfica.** A base não traz idade, etnia ou condição socioeconômica — **é impossível auditar viés entre subgrupos**, o que é uma lacuna séria para um sistema de saúde que atende população diversa.
- **Não é um dispositivo médico certificado.** Uso clínico real exigiria validação externa e prospectiva, aprovação regulatória (ANVISA), rastreabilidade de versão de modelo e monitoramento contínuo de *drift* e de viés.
- **Risco de excesso de confiança automática.** Um sistema que acerta 97% das vezes induz o profissional a relaxar a checagem — por isso a interface deve sempre expor a incerteza e a explicação, e nunca apenas o rótulo.

5.6.3 Próximos passos

- Validação externa em base independente e, idealmente, prospectiva.
- Coleta de variáveis demográficas para auditoria de viés entre subgrupos.
- Calibração explícita do limiar de decisão junto ao corpo clínico, com curva de custo-benefício definida pelos próprios médicos.

- Evolução do item de imagem (transfer learning, maior resolução, mais dados) para reduzir a dependência da extração manual de features.
- Ampliação do escopo para os outros riscos citados no desafio — SOP e detecção de sinais de violência doméstica em prontuários (NLP) — reaproveitando a arquitetura modular já construída.

6. EXTRA — Visão Computacional: CNN em imagens de ultrassom

O enunciado oferece como item extra o diagnóstico a partir de dados de imagem com redes neurais convolucionais. O projeto entrega esse pipeline completo — com treino, avaliação, escolha de ponto de operação e explicabilidade visual.

6.1 Arquitetura e treino

Aspecto	Escolha	Justificativa
Arquitetura	3 blocos Conv→BatchNorm→ReLU→MaxPool + AdaptiveAvgPool + Dropout + Linear	≈ 24 mil parâmetros: capacidade suficiente para 546 imagens de treino sem memorizar o conjunto
Entrada	64 × 64 px, 1 canal	Compromisso entre detalhe e custo; o `AdaptiveAvgPool` aceita outras resoluções sem mudar código
Aumentação	flip horizontal + rotação de ±10°	Amplia artificialmente um conjunto muito pequeno, com transformações plausíveis em ultrassom
Desbalanceamento	perda ponderada com expoente 0,5 sobre os pesos balanceados	Balanceamento total superprevê a minoria; a raiz suaviza e equilibra recall e precision
Seleção de época	melhor F1 de validação (checkpoint restaurado)	Early stopping implícito — evita entregar o modelo da última época por acaso
Hardware	CPU, ≈ 2–3 minutos	Reprodutível em qualquer máquina, sem GPU

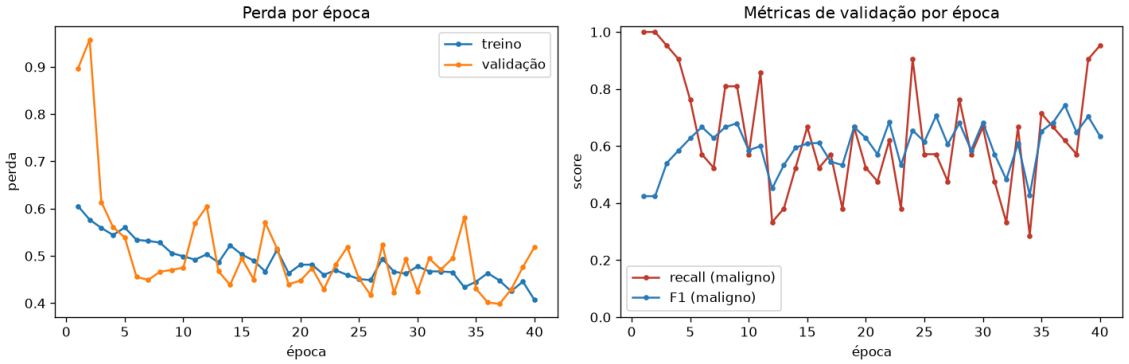


Figura 9 — Curvas de treino. À esquerda, a perda cai de forma consistente em treino e validação. À direita, recall e F1 de validação oscilam bastante entre épocas — consequência direta do conjunto de validação minúsculo (78 imagens), o que é exatamente o motivo de selecionar o melhor checkpoint em vez da última época.

6.2 Resultados e a escolha do ponto de operação

A CNN produz uma **probabilidade**, e o limiar que a converte em decisão é uma **escolha clínica, não estatística**. O projeto reporta dois pontos de operação — e o limiar clínico foi calibrado **no conjunto de validação, nunca no teste**, para não contaminar a estimativa final.

Ponto de operação	Accuracy	Precision	Recall	F1	ROC-AUC	Falsos negativos
Limiar padrão (0,50)	0,853	0,771	0,643	0,701	0,854	15 de 42
Ponto clínico (0,29)	0,718	0,485	**0,786**	0,600	0,854	**9 de 42**

Baixar o limiar de 0,50 para 0,29 **reduz de 15 para 9 os cânceres não detectados** em 156 imagens de teste, ao custo de mais alarmes falsos (precision cai de 0,771 para 0,485). É a mesma discussão de custo assimétrico da seção 5.4.1, agora aplicada à triagem por imagem — e em triagem essa é a troca correta. Note que o **ROC-AUC não muda** (0,854): ele mede a qualidade do ranqueamento, que independe do limiar. O

limiar apenas escolhe **onde** na curva ROC o sistema vai operar.

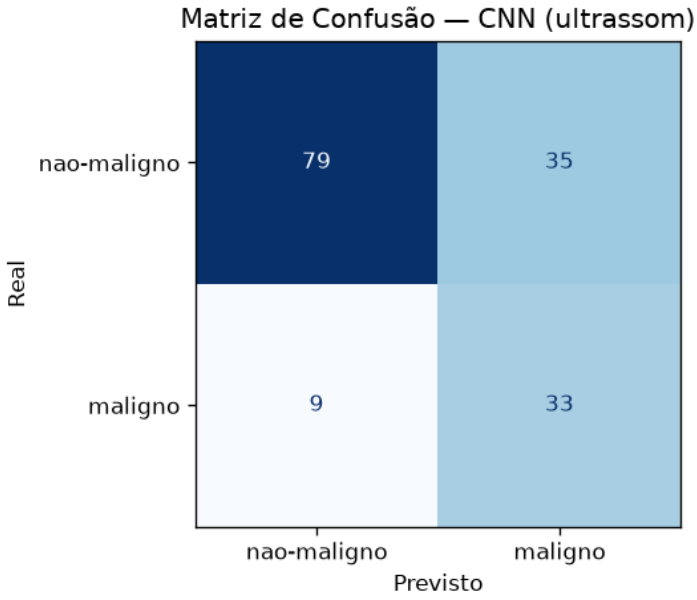


Figura 10 — Matriz de confusão da CNN no ponto de operação clínico: 33 dos 42 malignos detectados, 9 falsos negativos, 35 falsos positivos.

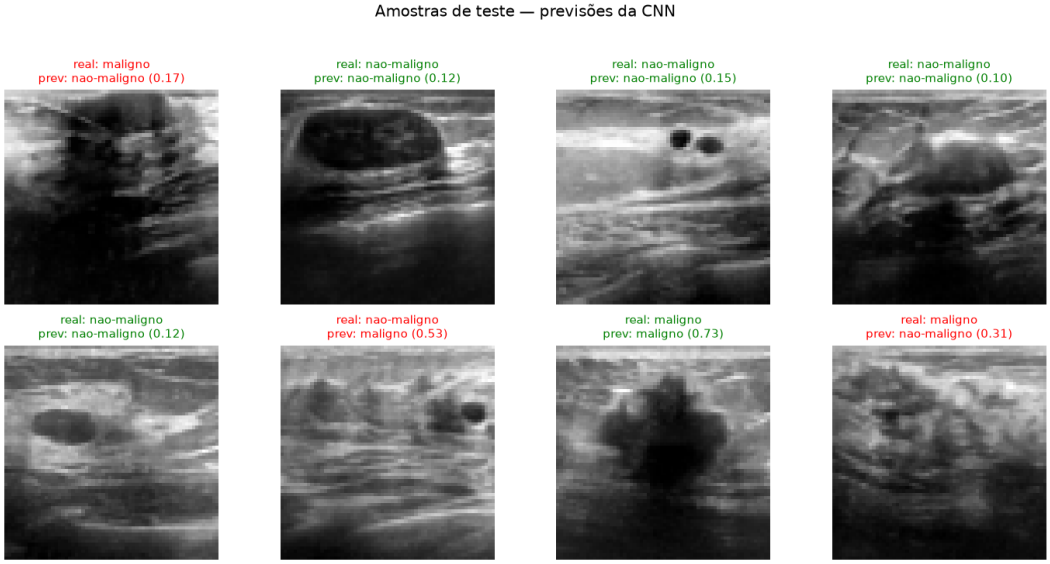


Figura 11 — Amostras do conjunto de teste com rótulo real e previsto (verde = acerto). As imagens são pequenas, ruidosas e de baixo contraste — o que ajuda a dimensionar a dificuldade da tarefa.

6.3 Explicabilidade visual — Grad-CAM

O Grad-CAM é o análogo visual do SHAP: ele usa os gradientes da classe “maligno” em relação à última camada convolucional para produzir um mapa de calor das regiões que mais pesaram na decisão. É o que permite **auditar** a rede antes de confiar nela.

Leitura crítica e honesta: em vários exemplos o mapa de calor aparece **difuso pela textura do tecido**, e não concentrado sobre o nódulo. Isso é um sinal legítimo de alerta — sugere que a rede está se apoiando em textura global e ruído de *speckle* em vez da lesão em si, o que é consistente com o AUC modesto de 0,85. **E esse é precisamente o valor do Grad-CAM:** sem ele, o número de accuracy sozinho poderia passar uma confiança que a evidência visual não sustenta.

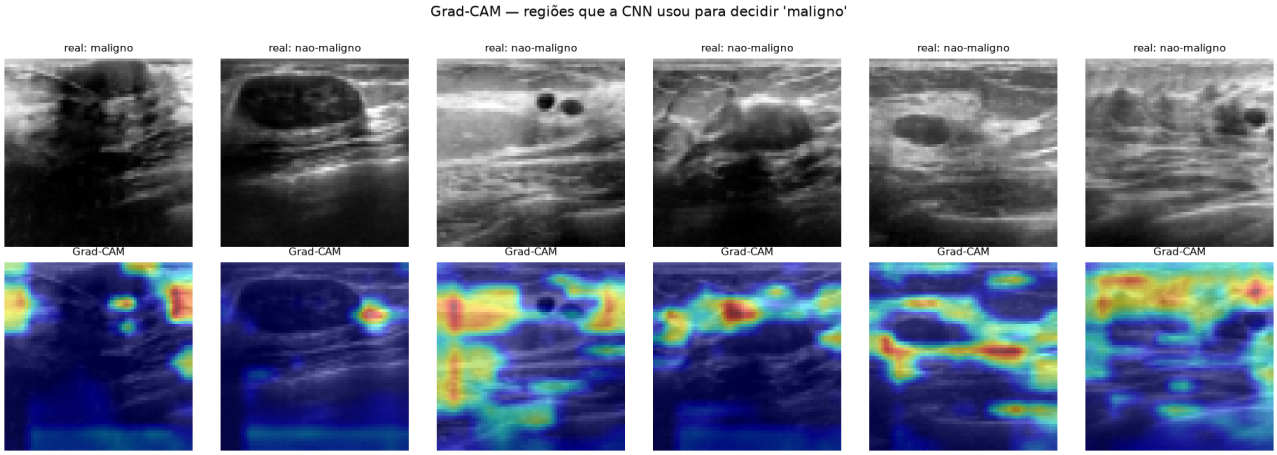


Figura 12 — Grad-CAM. Linha superior: imagem original de ultrassom. Linha inferior: o mapa de calor sobreposto (vermelho = maior influência na decisão “maligno”).

6.4 Leitura crítica do item EXTRA

O modelo de imagem é **honestamente mais fraco** que o tabular: AUC de 0,854 contra 0,995. Isso não é um defeito de implementação — é a consequência esperada de um problema muito mais difícil:

- O modelo tabular recebe **features que especialistas já extraíram** das lâminas; a CNN precisa **descobrir os padrões direto dos pixels**.
- São **546 imagens de treino** contra 455 amostras tabulares — mas o espaço de entrada é de 4 096 pixels em vez de 30 medidas curadas.
- As imagens são de **baixa resolução** (o BreastMNIST é derivado por redimensionamento) e ultrassom é uma modalidade intrinsecamente ruidosa.
- O valor de AUC ≈ 0,85 está **dentro do que a literatura reporta** para o BreastMNIST — não é um resultado anômalo.

O item EXTRA vale, portanto, como **prova de conceito e base de evolução**, não como sistema pronto. O caminho de melhoria é conhecido: **transfer learning** (ResNet/EfficientNet pré-treinadas), imagens em resolução original, mais dados e validação multi-institucional.

7. Provas de correção — a suíte de testes

Os resultados apresentados só valem se o encanamento de dados estiver correto. A suíte de testes existe para **provar as invariantes** das quais todos os números dependem — não para "ter testes".

Grupo	O que é provado	Por que importa
Integridade dos dados	569 × 31; alvo binário; 212 malignos e 357 benignos; nenhum ausente	Detecta troca silenciosa de base ou corrupção do CSV
Remapeamento do rótulo	1 = maligno em ambos os pipelines (tabular e CNN)	**Se estivesse invertido, todo o recall discutido estaria de cabeça para baixo** — e o erro seria invisível, pois as métricas continuariam "boas"
Split	Proporção de malignos preservada; zero sobreposição entre treino e teste	Garante que o teste é realmente inédito
Anti-vazamento (o mais importante)	A média aprendida pelo `StandardScaler` é idêntica à média do **treino** e mensuravelmente diferente da do dataset completo	**Se este teste falhar, todos os números de desempenho são inflados e inválidos**
Modelagem	Todos os modelos preveem apenas rótulos válidos; mesma semente produz previsões idênticas	Reprodutibilidade verificável, não prometida
Avaliação	Métricas dentro de [0,1]; TN+FP+FN+TP igual ao tamanho do teste	Impede erro aritmético silencioso na matriz de confusão
Explicabilidade	A importância cobre exatamente as 30 features, sem perda nem duplicação	Garante que o alinhamento nome-de-feature ↔ peso está correto
CNN	Saída binária; aceita 28/64/128 px; o limiar controla as predições nos dois extremos; Grad-CAM com shape correto e normalizado	Testes herméticos, sem download nem treino — rodam em segundos

```
python -m pytest -v
```

7.1 Execução verificada de ponta a ponta

A entrega foi validada em ambiente limpo (venv novo, `pip install -r requirements.txt` sem conflitos, Python 3.11 no Windows):

- **19 de 19 testes passam** (12 do pipeline tabular + 7 da CNN), incluindo o teste anti-vazamento e o teste de remapeamento do rótulo.
- `python main.py` **executa por completo** e reproduz exatamente as métricas relatadas: matriz de confusão do modelo campeão com **71 VN · 1 FP · 2 FN · 40 VP**.
- `python main_vision.py` **executa por completo** e reproduz o resultado da CNN de forma determinística: limiar clínico 0,29, matriz de confusão **[[79, 35], [9, 33]]** e ROC-AUC 0,854.
- Todas as **11 figuras** e os **2 CSVs** de métricas são regerados do zero pelos dois comandos — nenhum resultado deste PDF é digitado à mão.

8. Resultados obtidos — galeria completa de figuras

Todas as figuras abaixo são geradas automaticamente pelo pipeline e gravadas em `reports/figures/`. As principais já foram discutidas nas seções 5 e 6; esta seção reúne as que faltavam para o registro completo.

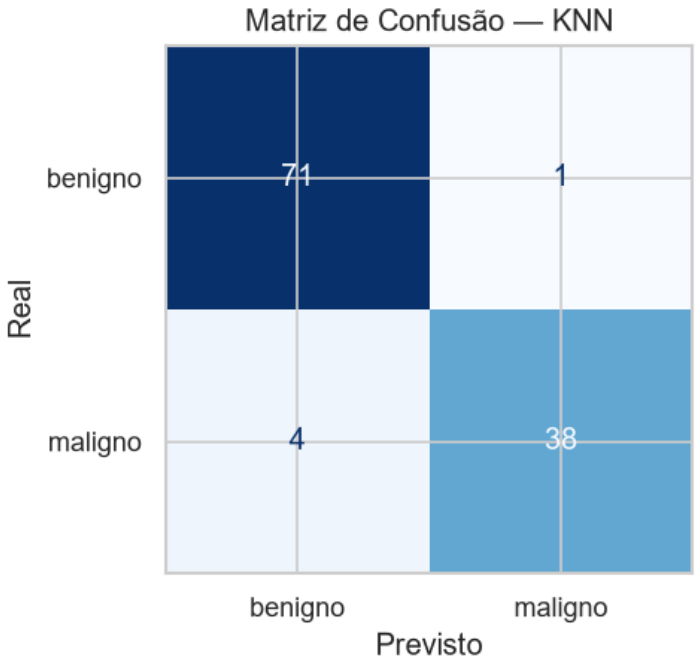


Figura 13 — Matriz de confusão do KNN (k = 7): 38 dos 42 malignos detectados, 4 falsos negativos.

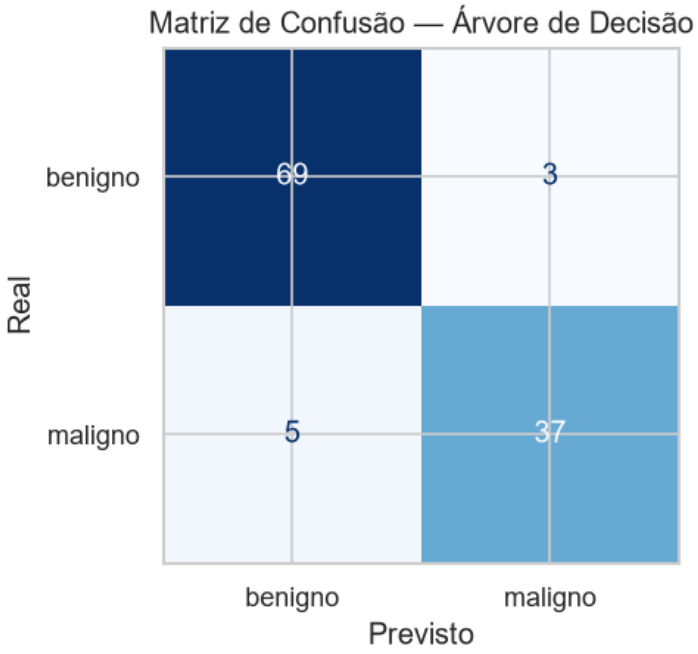


Figura 14 — Matriz de confusão da Árvore de Decisão (profundidade 4) — o modelo mais fraco do conjunto, com 5 falsos negativos e 3 falsos positivos.

8.1 Índice das figuras e onde cada uma é discutida

Arquivo	Conteúdo	Discussão
`01_balanceamento_classes.png`	Distribuição das classes	§5.1.1

Arquivo	Conteúdo	Discussão
`02_distribuicoes_por_classe.png`	Distribuições por classe (top 6 features)	§5.1.3
`03_matriz_correlacao.png`	Matriz de correlação de Pearson	§5.1.4
`04_matriz_confusao_*.png`	Matriz de confusão dos 4 modelos	§5.4.3 e §8
`05_curvas_roc.png`	Curvas ROC comparativas	§5.4.3
`06_feature_importance.png`	Importância global (coeficientes)	§5.5.1
`07_shap_summary.png`	SHAP summary plot	§5.5.2
`08_cnn_treino.png`	Curvas de treino da CNN	§6.1
`09_cnn_matriz_confusao.png`	Matriz de confusão da CNN	§6.2
`10_cnn_amostras.png`	Previsões em amostras de teste	§6.2
`11_cnn_gradcam.png`	Grad-CAM da CNN	§6.3

9. Conformidade com o enunciado

Mapa item-a-item entre as exigências do desafio e a entrega correspondente.

Exigência do enunciado	Onde está atendido
Dados e Modelos	
Escolher dataset público de saúde/segurança da mulher e discutir o problema	§2 deste PDF · `README` §1–2 · notebook §1
Exploração de dados	
Carregar a base e explorar suas características	§5.1 · `src/data_loader.py`, `src/eda.py` · notebook §2–3
Estatísticas descritivas e distribuições, com discussão	§5.1.2–5.1.3 · Figuras 1 e 2 · notebook §3.2–3.4
Identificar padrões de saúde e segurança feminina	§5.1.3 (padrão morfológico maligno) · notebook §3.4
Pré-processamento	
Limpeza, valores ausentes e inconsistentes	§5.2.1–5.2.2 · `src/preprocessing.py`
Pipeline de pré-processamento em Python	§5.2.3 · `ColumnTransformer` + `Pipeline`
Variáveis categóricas e numéricas em formato adequado	§5.2.4 — base 100% numérica; ramo categórico já plugável no `ColumnTransformer`
Análise de correlação	§5.1.4 · Figura 3 · `eda.highly_correlated_pairs()`
Modelagem	
Duas ou mais técnicas de classificação	§5.3 — **quatro**: Regressão Logística, Árvore de Decisão, KNN e Random Forest
Separação clara entre treino e teste	§5.2.5 — split estratificado 80/20 + CV de 5 folds só no treino
Treinamento e avaliação	
Treinar com o conjunto de treinamento	§5.3.1 · `main.py` etapa 4
Avaliar no teste com accuracy, recall e F1	§5.4.2 · `reports/metricas_teste.csv`
Discutir a escolha da métrica considerando o problema	§5.4.1 — custo assimétrico do erro, recall como prioridade
Interpretação com feature importance e SHAP	§5.5 · Figuras 7 e 8 · `src/explainability.py`
Discussão crítica: pode ser usado na prática? Como?	§5.6 — usos, limitações e próximos passos
O médico sempre tem a palavra final	§1 (premissa) e §5.6.1 — reafirmado ao longo de todo o documento
EXTRA — Visão Computacional	
Diagnóstico com dados de imagem usando CNN	§6 · `src/vision.py`, `main_vision.py`, notebook da CNN
Código e organização	
Projeto Python estruturado e documentado	§3 — pacote modular com docstrings, tipagem e testes
Notebook Jupyter ou scripts para demonstração	2 notebooks com saídas salvas + 2 scripts CLI
Entregáveis da Fase 1	
Arquivo PDF com link do repositório Git	Este documento — capa
Código-fonte completo	Repositório (§3)

Exigência do enunciado	Onde está atendido
README.md com instruções de execução	`README.md` · §4
Dockerfile (se usar docker)	`Dockerfile` + `.dockerignore` · §4.2
Dataset (ou link para download)	§2 — CSV versionado + download automático
Resultados obtidos (prints, gráficos e análises)	§5, §6 e §8 — 11 figuras + 2 CSVs de métricas
Relatório técnico (EDA, pré-processamento, modelos, resultados)	§5 completo
Vídeo de demonstração de até 15 min	§10 — roteiro; link na capa

10. Vídeo de demonstração

Link: https://youtu.be/XWGwS7_uU2o
Duração: até 15 minutos (alvo: 11–13 min) · **Visibilidade:** público ou não listado

O roteiro completo, com falas sugeridas e checklist de gravação, está em `docs/ROTEIRO_VIDEO.md` no repositório. Resumo do que é demonstrado:

Tempo	Bloco	O que é mostrado na tela
0:00–0:50	Abertura e problema	Contexto do desafio e a premissa do copiloto clínico
0:50–2:00	Dataset e discussão	`README` §2, o CSV e o bloco de convenção de rótulo em `config.py`
2:00–3:00	Arquitetura do projeto	Árvore de arquivos: `src/`, notebooks, testes, Dockerfile
3:00–4:30	Análise exploratória	Figuras 01, 02 e 03 — balanceamento, distribuições por classe e multicolinearidade
4:30–5:30	Pré-processamento	`preprocessing.py` e `modeling.py` — o scaler dentro do `Pipeline` (anti-vazamento)
5:30–7:30	**Pipeline rodando ao vivo**	`python main.py` no terminal, narrando as 7 etapas e a matriz de confusão detalhada
7:30–9:00	**Resultados e métrica**	Tabela de métricas, matriz de confusão do campeão e curvas ROC; discussão do custo assimétrico
9:00–10:15	**Explicabilidade**	Figuras 06 e 07 — coeficientes sob multicolinearidade e o SHAP que corrige a leitura
10:15–12:00	EXTRA — CNN	Notebook da CNN, os dois pontos de operação e o Grad-CAM com leitura crítica
12:00–12:45	Provas de correção	`python -m pytest -v`, com destaque para o teste anti-vazamento
12:45–13:30	**Discussão crítica**	`README` §7 — uso prático, limitações e a palavra final do médico

Antes de gravar: rode `python main.py` e `python main_vision.py` uma vez, para que as figuras estejam prontas e o BreastMNIST já esteja em cache — o download do dataset não deve aparecer no vídeo.

Anexo A — Dataset e reprodutibilidade

Item	Detalhe
Base principal	Breast Cancer Wisconsin (Diagnostic) — 569 × 30
Como obter	Automático via `sklearn.datasets.load_breast_cancer()`; cópia em `data/breast_cancer_wisconsin.csv`
Link público	kaggle.com/datasets/uciml/breast-cancer-wisconsin-data · UCI ML Repository
Base EXTRA	BreastMNIST (MedMNIST v2), 780 imagens de ultrassom
Como obter	Automático via pacote `medmnist` (download do Zenodo)
Link público	medmnist.com · Zenodo
Semente	`RANDOM_STATE = 42`, definida em `src/config.py` e usada em split, CV, modelos e treino da CNN
Ambiente	Python 3.11; versões fixas em `requirements.txt`; imagem Docker disponível
Observação	Pequenas variações numéricas podem ocorrer entre versões de biblioteca; no mesmo ambiente os resultados são determinísticos

Anexo B — Referências

- Wolberg, W. H., Street, W. N. & Mangasarian, O. L. (1995). *Breast Cancer Wisconsin (Diagnostic) Data Set*. UCI Machine Learning Repository.
- Yang, J. et al. (2023). *MedMNIST v2: A large-scale lightweight benchmark for 2D and 3D biomedical image classification*. Scientific Data, 10(1).
- Al-Dhabyani, W., Gomaa, M., Khaled, H. & Fahmy, A. (2020). *Dataset of breast ultrasound images*. Data in Brief, 28.
- Lundberg, S. M. & Lee, S.-I. (2017). *A unified approach to interpreting model predictions*. NeurIPS.
- Selvaraju, R. R. et al. (2017). *Grad-CAM: Visual explanations from deep networks via gradient-based localization*. ICCV.
- Pedregosa, F. et al. (2011). *Scikit-learn: Machine learning in Python*. JMLR, 12.

Declaração de uso responsável. Este projeto é um trabalho acadêmico. O sistema descrito **não é um dispositivo médico certificado** e não deve ser utilizado para decisão clínica sem validação externa, aprovação regulatória e supervisão profissional. Toda saída do modelo é uma sugestão acompanhada de explicação — **a decisão diagnóstica é sempre do médico**.